

```
/tikz/,/tikz/graphs/  
conversions/canvas coordinate/.code=1 , conversions/coordinate/.code=1  
layered
```

Max Cut:

Deep Learning vs. Heuristic Algorithms

Victor Pekkari Armon Haghghi
[vpekkari@ucsd.edu] [ahaghghi@ucsd.edu]

University of California, San Diego

June 2025

1 Introduction

The aim of this project is to systematically compare *classical* Max-Cut solvers with *learned* neural approaches. We focus on three guiding questions: (i) Can deep networks match or surpass the solution quality of state-of-the-art heuristics and approximation algorithms like Geomans-Williamson? (ii) How does each method’s performance vary with graph size, edge density, and structural class? and (iii) What constitutes a fair evaluation when only some algorithms enjoy provable guarantees while others can only be assessed only empirically? If neural networks can be used as an alternative, what graphs do they work the best on? and What networks are the best for solving maximum cut.

All code and experiments for this experiment can be found in the following GitHub repo: https://github.com/victor99pekk/ML_4_MAXCUT

2 Maximum Cut

A maximum cut divides a graph’s vertices into two groups. It aims to maximize the number or the total weight of edges crossing between those groups.

Because the Maximum-Cut problem is NP-hard, exact algorithms that scale to large instances are computationally infeasible, making approximation methods indispensable. The effectiveness of a given algorithm may depend on the structure, size, and density of the graph at hand, and the method that excels on sparse social networks may falter on dense weighted meshes. Evaluating how al-

algorithm choice should adapt to different graph classes is therefore as important as knowing their theoretical guarantees. For example, the Goemans–Williamson algorithm offers a worst-case guarantee of 0.878 of the optimal cut, but understanding how close it gets in practice on specific graph families remains an essential empirical question in order to compare to neural networks where we can only get an empirical estimation of the performance. It is not fair to compare the empirical performance of one algorithm to the theoretical lower bound of another.

2.1 Definition

A definition of the maximum-cut problem can be given as follows:

Definition 2.1 (Maximum Cut). Let $G = (V, E, w)$ be an undirected, possibly weighted, graph with vertex set $V = \{1, \dots, n\}$, edge set $E \subseteq \binom{V}{2}$, and non-negative weights w_{ij} for every edge $\{i, j\} \in E$ (take $w_{ij} = 1$ for the unweighted case).

For any subset $S \subseteq V$ define the *cut set*

$$\delta(S) = \{\{i, j\} \in E \mid i \in S, j \in V \setminus S\},$$

and its *cut value*

$$\text{cut}(S) = \sum_{\{i, j\} \in \delta(S)} w_{ij}.$$

The Max-Cut problem consists of finding the subset $S^* \subseteq V$ that maximises this value:

$$\text{MaxCut}(G) = \max_{S \subseteq V} \sum_{\{i, j\} \in E} w_{ij} \mathbf{1}_{[i \in S, j \notin S]}.$$

An equivalent form, the binary quadratic form is, $x \in \{-1, 1\}^n$ defined by $x_i = 1$ if $i \in S$ and $x_i = -1$ otherwise, the problem can be written as

$$\text{MaxCut}(G) = \max_{x \in \{-1, 1\}^n} \frac{1}{2} \sum_{\{i, j\} \in E} w_{ij} (1 - x_i x_j).$$

2.2 Computational Complexity

The computational difficulty of Max-Cut comes from the size of its search space. For a graph on n vertices there are $2^{n-1} - 1$ bipartitions, so a brute force algorithm, that checks every possible partition already needs $\Theta(2^n)$ time. This is the characteristic of many NP-hard problems. As a consequence of this, any exact algorithm must in the worst case explore an exponential subset of those 2^{n-1} possibilities, and the fastest known exact methods such as branch-and-bound still run in $O^*(2^{\alpha n})$ for some $\alpha < 1$ and are still exponential. This complexity explains the need of approximation techniques such as semidefinite relaxations,

randomised hyperplane cuts, and, learning-based heuristics. These all trade provable optimality for polynomial-time performance while trying to stay as close as possible to the optimum that a 2^n -state search would give.

2.3 Lower Bounds

Despite the complexity of finding an optimal cut, every graph has a provably large cut, given by the following equations.

$$\text{MaxCut}(G) \geq \left\lfloor \frac{m}{2} + \sqrt{\frac{m}{8} + \frac{1}{64}} - \frac{1}{8} \right\rfloor,$$

and, if G is connected, the Edwards–Erdős bound gives,

$$\text{MaxCut}(G) \geq \frac{m}{2} + \frac{n-1}{4}.$$

For signed graphs, with each edge labelled $+$ or $-$, the bound gives a minimum number of balanced edges, and for weighted graphs Poljak and Turzík proved

$$\text{MaxCut}(G) \geq \frac{w(G)}{2} + \frac{w(T_{\min})}{4},$$

where $w(G)$ is the total edge weight and $T_{\min}$ is any minimum-weight spanning tree. These guarantees are a good baseline that any heuristic or learning based solver should at least match and/or beat.

3 Heuristic approximation algorithms

The heuristic algorithms that have shown to be useful for solving Maximum Cut have been implemented by first relaxing the NP-hard problem into something more feasible. For the approaches we used, the Geomans-Williamson and some variations, by substituting variables to vectors, and optimizing over a semi-definite matrix, we can instead solve a semi-definite program, *SDP*, that will give us an approximate solution for the maximum cut. The heuristic methods are still, to this day, the best mathematically proven approximations that provide a guaranteed lower bound.

3.1 Theorems

Definition 3.1 (Semidefinite program). Given symmetric matrices $A, B_1, \dots, B_m \in \mathbb{R}^{n \times n}$ and scalars $b_1, \dots, b_m \in \mathbb{R}$, the *semidefinite programme* is

$$\max \langle A, X \rangle \quad \text{s.t.} \quad X \succeq 0, \langle B_i, X \rangle = b_i \quad (i = 1, \dots, m),$$

where $\langle A, X \rangle = \text{tr}(A^\top X)$. The region is convex and therefore the programmes are solvable in polynomial time. first-order methods.

Interpretation. This abstract template captures every SDP we use: the Max-Cut relaxation simply sets B_i such that $\text{diag}(X) = 1$ and maximises a weighted sum of $1 - X_{ij}$.

Theorem 3.2 (Quality of the SDP relaxation [?, Thm. 3.5.6]). *For any symmetric positive-semidefinite matrix A let*

$$\text{INT}(A) = \max_{x \in \{\pm 1\}^n} \sum_{i,j} A_{ij} x_i x_j, \quad \text{SDP}(A) = \max_{\substack{X \succeq 0 \\ X_{ii}=1}} \sum_{i,j} A_{ij} X_{ij}.$$

Then

$$\text{INT}(A) \leq \text{SDP}(A) \leq 2K \text{INT}(A),$$

where $K \approx 1.783$ is the Grothendieck constant.

Interpretation. Any integer quadratic problem can be relaxed to an SDP whose optimum overestimates the true optimum by at most a universal factor $2K < 3.6$; for Max-Cut this gives a non-trivial upper bound even before rounding.

Theorem 3.3 (0.878-approximation for MAX-CUT [?, Thm. 3.6.5]). *Let G be a graph with adjacency matrix A and let $(X_i)_{i=1}^n$ be optimal vectors for the SDP*

$$\frac{1}{4} \max \left\{ \sum_{i,j} A_{ij} (1 - \langle X_i, X_j \rangle) : \|X_i\|_2 = 1 \right\}.$$

Sample a random hyperplane by $g \sim \mathcal{N}(0, I)$ and set $x_i = \text{sign}\langle X_i, g \rangle$. Then

$$\mathbb{E}[\text{CUT}(G, x)] \geq 0.878 \text{SDP}(G) \geq 0.878 \text{MAX-CUT}(G).$$

The Goemans–Williamson rounding converts any SDP solution into a cut which has expected weight is at least 87.8% of the optimum.

3.2 Relaxation of Max-Cut

The idea behind relaxing Max-Cut is to transform binary constraints into vector constraints. This allows continuous optimization methods to solve the problem. We relax the condition from discrete points $\{-1, 1\}$ to points on a sphere. Starting from the binary quadratic form

$$\text{MaxCut}(G) = \max_{x \in \{-1, 1\}^n} \frac{1}{2} \sum_{\{i,j\} \in E} w_{ij} (1 - x_i x_j) = \max_{x \in \{-1, 1\}^n} \frac{1}{4} x^\top L x,$$

where L is the weighted Laplacian with entries $L_{ii} = \sum_j w_{ij}$ and $L_{ij} = -w_{ij}$ for $i \neq j$, we perform two algebraic lifts that convert the NP-hard 0–1 programme into a convex optimisation.

We start with replacing every scalar variable $x_i \in \{-1, 1\}$ by a unit vector $v_i \in \mathbb{S}^{n-1}$ and substitute $x_i x_j \rightarrow \langle v_i, v_j \rangle$. The objective becomes

$$\max_{v_1, \dots, v_n \in \mathbb{S}^{n-1}} \frac{1}{2} \sum_{\{i,j\} \in E} w_{ij} (1 - \langle v_i, v_j \rangle).$$

We then collect the inner products in a Gram matrix $X \in \mathbb{S}^n$ via $X_{ij} := \langle v_i, v_j \rangle$. By construction $X \succeq 0$ and $\text{diag}(X) = \mathbf{1}$. The relaxation therefore is,

$$\begin{aligned}
(\text{SDP}) \quad & \max_{X \in \mathbb{S}^n} \quad \frac{1}{4} \sum_{i,j=1}^n w_{ij} (1 - X_{ij}) \\
\text{s.t.} \quad & X_{ii} = 1 \ (1 \leq i \leq n), \quad X \succeq 0.
\end{aligned} \tag{3.1}$$

3.3 Goemans-Williamson Algorithm

Algorithm 1 Goemans–Williamson Approximation for MAX–CUT

Require: Graph $G = (V, E)$, rounds R (default $R = 10$)

Ensure: Cut $(S, V \setminus S)$

```

1: Solve SDP to get Gram matrix  $X^*$ 
2: Factor  $X^* = VV^\top$ ; let  $v_i$  be row  $i$  of  $V$  ▷ each  $v_i \in S^{n-1}$ 
3: bestWeight  $\leftarrow 0$ , bestCut  $\leftarrow \emptyset$ 
4: for  $r \leftarrow 1$  to  $R$  do
5:   Sample  $g \sim \mathcal{N}(0, I)$ 
6:    $S \leftarrow \{i \in V : \langle v_i, g \rangle \geq 0\}$ 
7:    $w \leftarrow \frac{1}{2} \sum_{\{i,j\} \in E} \mathbf{1}[(i \in S) \neq (j \in S)]$  ▷ cut weight
8:   if  $w > \text{bestWeight}$  then
9:     bestWeight  $\leftarrow w$ , bestCut  $\leftarrow S$ 
10:  end if
11: end for
12: return (bestCut,  $V \setminus \text{bestCut}$ )

```

3.4 Implementation

Our Goemans–Williamson code that used two libraries, `CVXPY` (with the first-order solver `SCS`) for the semidefinite optimisation. For each row of the input file the script reconstructs the weight matrix, solves the Max-Cut SDP at relative tolerance 10^{-6} , factorises the resulting Gram matrix with `numpy.linalg.eigh`, and evaluates ten random hyperplane roundings, keeping the heaviest cut.

4 Neural Network Models

All networks that we tried were supervised and processed the adjacency matrices sequentially, one row at a time. We first tried a LSTM encoder-decoder network inspired by Gu and Yang [2] approach to the problem, We then tried to replace the LSTM cells with a more modern transformer approach instead. Since both of our networks were supervised and processed the input sequentially, we will describe how these supervised recurrent neural networks work in sections 4.1–4.4, these sections can be skipped if the reader already knows the topics RNN, LSTM, transformers, and their timecomplexity.

4.1 Recurrent Neural Network

RNN:s are the simplest form of neural networks that can handle sequential data, RNN stands for recurrent neural network. The RNN "remembers" the previous input by keeping a hidden state h_t and for every input x_t in the sequence $x_{1:T}$

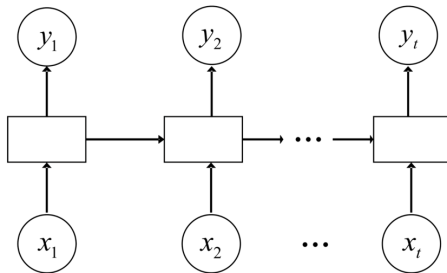


Figure 1: unfolded RNN

$$h_t = \sigma(W_x \cdot x_t + W_h \cdot h_{t-1})$$

$$y_t = W_y \cdot h_t$$

it produces one output y_t . Since we feed the RNN a sequence of $x_{1:T}$ it will also output a sequence of outputs $y_{1:T}$ while keeping one hidden state that it continuously updates after every new input x_t form the input sequence. We can see that as we unfold our RNN as in the figure 1 that the depth of our network grows $\mathcal{O}(n)$ with the sequence length, this means we will have to multiply more numbers in order to do backpropagation (because of the chain rule), multiplying several numbers like this have a risk of leading to an exploding or vanishing gradient. This sensitivity of depth is widely known as the biggest flaw with the vanilla-RNN. In order to solve the problem of vanishing/exploding gradient, the LSTM was invented.

4.2 Long short-term term memory

LSTM:s solve the problems of exploding/vanishing gradient by introducing a separate memory cell C_t alongside the hidden state h_t , and by using gating mechanisms to control what information is stored, forgotten, and outputed at each time-step. But LSTM:s still work on the same idea as the RNN, by taking a sequence of input, keeping some hidden state, and producing a sequence of outputs. For each input x_t in the sequence $x_{1:T}$, the LSTM maintains both a cell state C_t and a hidden state h_t . Sometimes the most relevant output from the LSTM is the very last output¹, and sometimes the whole output sequence is important ²

¹this is the case in our Encoder of our LSTM-model

²this is the case in the decoder of our LSTM-model

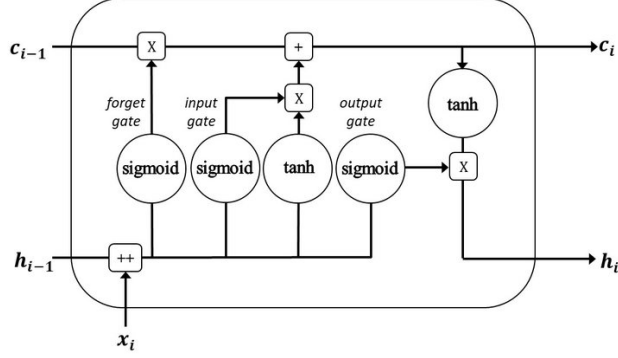


Figure 2: LSTM Cell

By explicitly maintaining the cell state C_t and using these gates, the LSTM can decide at each time step how much of the past information to carry forward (via f_t), which new information to write (via i_t and $\tilde{C}_t$), and what to expose as the current hidden representation (via o_t).

$$\begin{aligned}
f_t &= \sigma_g(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i) \\
o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o) \\
\tilde{c}_t &= \sigma_c(W_c x_t + U_c h_{t-1} + b_c) \\
c_t &= f_t \circ c_{t-1} + i_t \circ \tilde{c}_t \\
h_t &= o_t \circ \sigma_h(c_t)
\end{aligned}$$

4.3 Transformers

Transformers further improve on the idea of having sequential input data, and generating sequential output data. Transformers removes the explicit recurrence altogether and instead rely on *self-attention* to let every position in the input sequence exchange information with every other position in a single, constant-depth layer. Each input token³ $x_t \in \mathbb{R}^{d_{\text{model}}}$ is linearly projected to a *query*, *key*, and *value*. By representing the input-tokens with learnable key-, query- and value vectors, we can represent what a certain input-token is looking for (query), what it represents (key), and lastly what context it provides (value) at the same time.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k},$$

³In our Max-Cut setting a “token” is a node, represented by its adjacency-row fingerprint introduced in the previous subsection.

and the pairwise compatibility between tokens is measured by the *scaled dot-product*

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

This produces a new representation in which each node is a weighted sum of *values* from every other node, with weights determined by how strongly its *query* matches their *keys*. By stacking several of these “attention heads” in parallel (*multi-head attention*), followed by a position-wise feed-forward layer, a Transformer encoder layer can model long-range dependencies without the vanishing-gradient bottleneck that plagued vanilla RNNs and motivated LSTMs.

Why Transformers for Max-Cut? Compared with the LSTM encoder–decoder above, a Transformer offers three concrete advantages:

1. **Parallelism.** All nodes are processed simultaneously, cutting training time from $\mathcal{O}(n)$ per layer to $\mathcal{O}(1)$ on GPU/TPU hardware.
2. **Global context in every layer.** Unlike an LSTM where each input can only interact with a summarization of past only past tokens and cannot interact with future inputs, a single encoder self-attention layer in a Transformer lets *every* token directly attend to *all* other tokens (past and future) in the sequence, and even the transformer decoder let’s every input token directly interact with all past tokens. This full pairwise interaction makes it far easier to capture non-local structures⁴

4.4 Inference and Complexity — Transformer and LSTM

The computational complexity and the inference time are two very important factors in this project since the whole purpose is to find a way to either find the maxcut faster than current approximation algorithms, or a more optimal cut.

4.4.1 Training

The less important part of a neural networks complexity is the training complexity. If we can choose to spend more time training a neural network in exchange for lower inference times we want to do it. We won’t analyze the training complexity in this paper but rather focus on the complexity comparable to that of heuristic approximation algorithms.

4.4.2 Inference

Transformers: The dominating term in the timecomplexity for transformers occur in the self-attention, which consist of matrix mulitplications when we compute attention scores between all tokens. Calculating this consist of 2 matrix multiplications ($Q \cdot K^T$ followed by $\text{Attn} \cdot V$). The naive way for multiplying

⁴The context of other rows in the adjacency matrix is crucial for understanding the meaning of a row in our case

matrices has a complexity of $\mathcal{O}(n^3)$. You might think that this would mean that attention has a timecomplexity of $\mathcal{O}(n^3)$, but the timecomplexity is in fact (for one layer):

$$\mathcal{O}(n^2 \cdot d) \xrightarrow{\text{assuming } n > d} \mathcal{O}(n^2)$$

(d is embedding size and n is the number of inputs)

This said, a GPU can drastically speed this up by calculating the parts of the matrix multiplications in parallel.

LSTM: The heaviest computations in the LSTM cell are the multiplications between the weight matrices and the input vectors, before we apply the sigmoid function $W \cdot x$. The dense products cost:

$$W_x \cdot x_t \implies \mathcal{O}(d_x d_h) \quad W_h \cdot h_{t-1} \implies \mathcal{O}(d_h^2)$$

After removing all terms that aren't the heaviest we get the following timecomplexity for on LSTM cell for n timesteps:

$$\mathcal{O}(n(d_x d_h + d_h^2))$$

The matrix-vector multiplications can be speed up by a GPU the same was as the matrix multiplications in the transformer, but the linear increase with the input sequence length remains.

Take-away: Both models benefit substantially from GPU acceleration, yet their scaling differs. Transformers are quadratically dependent on sequence length but fully parallelisable, whereas LSTMs are linearly dependent on sequence length but sequential across time-steps. For long sequences on GPU-hardware, a Transformer layer is often faster in practice despite its higher $\mathcal{O}(n^2)$ arithmetic cost. But if we do have a shorter input sequence it is possible that a LSTM model will be faster.

4.5 Input to Network

The training data to all networks we tried consisted of a graph represented by an adjacency matrix, along with a partition of the nodes leading to the maximum cut.

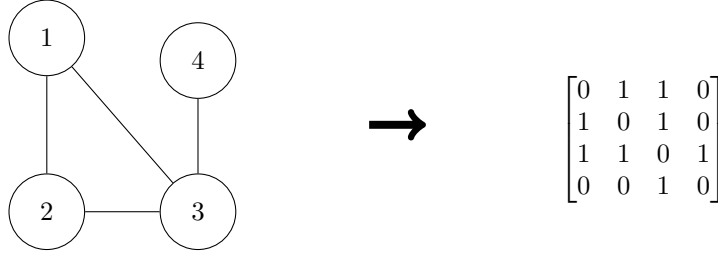
Every node in the graph can be viewed as being represented by it's row in the adjacency matrix, it is intuitive to see the row_i as that $node_i$'s fingerprint since it is often (almost always) a unique set of nodes it is connected to. It is $\sum_n^N \frac{1}{2^n}$ that another node has the same fingerprint⁵. This probability becomes vanishingly small fast.

⁵N is number of nodes in the graph

4.5.1 Graph encoding with adjacency matrices

We encode the graphs with adjacency matrices where every edge is encoded as a 1, and every absence of a edge between two nodes is encoded as 0.

An example of a graph with 4 nodes and it's adjacency matrix:



$$\mathbf{target} = [1, 0, 0, 1]$$

The target for this graph is the partition of nodes into two groups that yields the maximum cut. For this example the **target** is a vector of length 4.

4.5.2 Generating training data

Since finding the partition corresponding to the maxcut is NP-hard we cant really generate random adjacency matrices and then brute force them for the maxcut (target for the model we train) to generate training data. We also cannot use a heuristic approximation algorithm for calculating the target since we then would not be training our model on the most optimal target.

To solve these problems, we follow the approach in the already mentioned paper by Gu and Yang [2] for generating the training data. The method involves planting a solution by producing a quadratic objective matrix Q and a ground-truth binary vector $x \in \{-1, 1\}^n$ such that x is guaranteed to be the maximizer of the associated binary quadratic form for the quadratic programming problem. It has been shown that any quadratic programming problem can be rewritten as a maximum cut problem⁶[3] and this is exactly what we aim to do. The algorithm for doing this can be found below. We only managed to create adjacency matrices with entries having 25% chance of being a 1⁷. We generated both our test data and training data with seed 0 (for reproducibility) and the file `gen_maxcut_data.py` in the projects repository.

⁶This means that the planted solution in the quadratic programming problem is also going to be the optimal solution to the maximum cut

⁷These matrices proved to be too easy for the approximation algorithm to solve

Algorithm 2 A benchmark generator for the MAX-CUT problem

Require: Dimension n ; **base** = 10

Ensure: Matrix Q ; Vector x

- 1: Generate an $n \times n$ matrix Q with i.i.d. entries from standard normal
 - 2: $Q \leftarrow \text{base} \times Q$
 - 3: $Q \leftarrow \frac{1}{2}(Q + Q^\top)$ $\triangleright$ make Q symmetric
 - 4: Generate $x \in (0, 1)^n$ uniformly at random
 - 5: $x \leftarrow 2x - 1$
 - 6: Compute $\lambda \leftarrow \sum \text{abs}(Q)$ row-wise
 - 7: Form Q' with λ on the diagonal and zeros elsewhere
 - 8: $c \leftarrow (Q + Q') \cdot x$
 - 9: Set $w_{0j}, w_{oj} \leftarrow \frac{1}{4} \left(\sum_{j=1}^{i-1} q_{ji} + \sum_{j=i+1}^n q_{ij} \right) + \frac{1}{2}c_i, \quad 1 \leq j \leq n$
 - 10: Set $w_{ij} \leftarrow \frac{1}{4}q_{ij}, \quad 1 \leq i < j \leq n$
 - 11: Update $Q \leftarrow (q_{1j}^\top, q_{2j}^\top, \dots, q_{(n+1)j}^\top) \leftarrow (w_{0j}^\top, w_{1j}^\top, \dots, w_{nj}^\top)$
 - 12: Set $x_0 \leftarrow 1$ and update $x \leftarrow 2x + 1$
 - 13: Update $x \leftarrow (x_1, x_2, \dots, x_{n+1}) \leftarrow (x_0, x_1, \dots, x_n)$
-

Model 1: LSTM Model

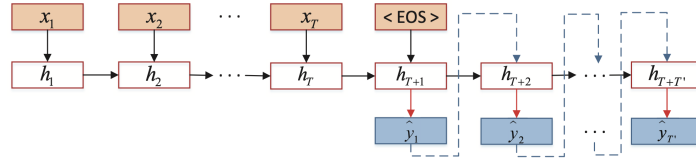


Figure 3: picture from Gu and Yang paper [2]

We use two LSTM cells in our encoder-decoder network, one as an encoder and one as a decoder. It can be seen in Figure 3 how we input our rows as a sequence to get the hidden states $h_1, \dots, h_{r+1}$ generated by the first encoder. The hidden state h_{r+1} is passed to the second decoder where it serves as start-hidden state, the decoder then generates the output sequence $y_1, \dots, y_r$ after first being fed a special start token. The hidden state that we pass from the encoder-LSTM to the decoder-LSTM can be seen as an encoding of the adjacency matrix. The decoder first outputs all nodes in the first partition, followed by a special EOS token, followed by the rest of the nodes that are in the second partition.

Token Embedding: We don't really have any traditional embedding. This is because the number of possible permutations of the binary row vector gets too big as n increases ($\mathcal{O}(2^n)$). We do have a linear layer that expands the number of columns in the row, to the embedding dimension (which we vary throughout the experiments).

Outputs, decoder: The outputs from the decoder represents a vector of the

same length as the number of nodes in the graph plus one for the separation token. index i represents to the probability that node i is the next node to be selected as the output. We mask all previously chosen nodes so they don't get chosen more than once.

Model 2: Transformer Model

Inspired by the success of Transformer architectures in capturing global structure, we try to further improve the LSTM pointer network from Gu and Yang paper [2] with a fully Transformer-based variant. Our transformer model consists of:

Encoder. A stack of 4 Transformer encoder layers jointly processes all n node embeddings in parallel. We use 4 layers of of encoding attention when we create our encoding block.

Decoder. A stack of 2 Transformer decoder layers generates one pointer at a time. We start from a special start token (EOS), and at each step the decoder attends over the full encoder memory to produce a new decoder hidden state.

Pointer head. At each decoding step, we compute a dot-product between the latest decoder output and each encoder memory vector to obtain logits over the n input nodes. After applying softmax, this yields a probability distribution for selecting the next node in the output partition.

In effect, the Transformer encoder captures *all* pairwise interactions among nodes in each layer, and the Transformer decoder learns to “point” into this global memory to produce the cut partition. This design removes recurrence entirely, enabling full parallelism over the node set during both encoding and (up to masking) decoding, and letting training and inference leverage highly optimized matrix multiplications on modern hardware.

5 Experiments

The Experiments we made were between the geomans-williamson, a LSTM neural network, and a transformer.

5.1 Neural network model

Training: The two PyTorch models were trained using the same Python script with the only difference being what model we trained, with the only difference being the model created. The training was done on a T4 GPU by running the code on Google Colab. The training was run for 30 minutes or until convergence, this was because the access to the GPUs on Google Colab is limited, so we didn't want to waste GPU time on non-increasing models. This was a big flaw in our

training since the increase in performance during training can plateau to then continue increasing, or just continue increasing at a really slow pace.

Inference: The test inference was done on a CPU, this was because lack of access to GPUs. We used Google Colab to access GPUs, but since the files containing the weights were very big we couldn't upload the biggest network-weights to Google Colab. We furthermore wanted to compile that models for increased inference speed⁸, and the only compilation with the standard PyTorch library was not compatible with python3 which was the only available python version on Google Colab. The estimated inference speed in the experiment results are estimated inference times on a GPU by running executing the models on CPUs, all tests where run on a CPU for the neural network. Since it is more relevant to know the inference speed on a GPU than a CPU, we decided to calculate the estimated inference time on GPU and compare that time with the inference time for the Geoman-Williamson's. We base our estimation from a recent paper [1] comparing inference speed between CPUs and GPUs on convolutional neural networks (which are highly parallisable, probably more so than LSTM at least, but still relevant in my opinion) and matrix-vector multiplication. The study showed that inference can be up to 4-5x faster on GPUs, so to get the estimated GPU inference times (in the results) we divide by 3.

5.2 Evaluation

5.2.1 Metrics

When finding the maximum cut, there are two things we are concerned with. How fast can we do it, and how close can we get to the maximum cut. This is therefore the two metrics we use to evaluate the algorithms.

Cut quality. Every solver outputs a partition, so our primary comparison is the *achieved cut weight*. We measure the achieved cut weight by dividing it by the maximum cut, this way we know how much better we can be, or if we are close. It also makes it easy to compare different models with this metric.

Inference speed. This is measured as the execution time for solving 1000 examples of graphs in five different sizes.

5.2.2 Benchmarks & data

For each of the following experiments, we produced training data, 1000 graphs for $n=5,10,20,30,50$ and took an average result of our findings.

⁸Compiling the models led to a 2x increase in inference speed

⁹Runtime for neural networks is an estimate, see [1].

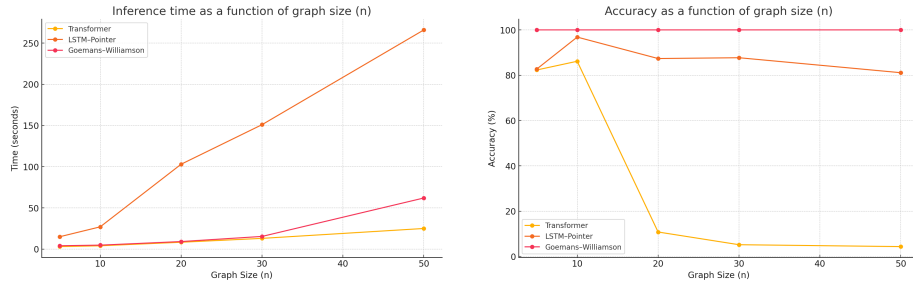
Table 1: Metrics, where n is the number of nodes in the graph

Metric	Transformer					LSTM-Pointer				
	5	10	20	30	50	5	10	20	30	50
Train loss	0.48	0.72	2.22	2.72	3.86	0.35	0.01	0.11	0.03	0.19
Test accuracy (%)	82.32	86.17	10.79	5.18	4.34	82.72	96.84	87.37	87.73	81.12

Table 2: Accuracy and total runtime⁹

for solving all 1000 graphs.

n	Transformer		LSTM-Pointer		Goemans-Williamson	
	Acc. (%)	Time (s)	Acc. (%)	Time (s)	Acc. (%)	Time (s)
5	82.32	3	82.72	15	100	4.05
10	86.17	4.0	96.84	27	100	4.80
20	10.79	8.3	87.37	103	100	9.13
30	5.18	13	87.73	151	100	15.36
50	4.34	25	81.12	266	100	62.00



5.3 Summary of Results

1. **Solution quality.** Goemans-Williamson get's *100 %* of the optimum on every instance we generated, whereas the LSTM-Pointer network stays in the 82-97 % band for $n \leq 30$ and dips to 81 % at $n = 50$. The Transformer starts with (≈ 83 -86 % for $n \leq 10$) but completely collapses once $n \geq 20$, giving less than 6 % accuracy for $n = 30$ and 50.
2. **Runtime.** GW completes in ≤ 62 ms even for the largest graphs. The LSTM was as expected from theory a lot slower than the transformer even though we estimed their GPU runtime the same wasy by dividing by 3 (even though the transformer is more parallelisable). The transformer is the fastest one.
3. **Training cost.** The models were not trained intensive, at most for 30 minutes on a GPU.
4. **Graphs:** When generating graphs we gave 25% chance of node A and

node B of being connected for all nodes where A is not B. This partly because we struggled with generating other data that had the property of having 0 and 1 as entries in the adjacency matrix, and we felt we had to get started with the experimenting. We first did not think it would matter very much if nodes were being connected with 50% as in the paper [2] or 25% which was what we had. But it turned out that our sparser networks were very easy for the approximation algorithm to solve.

5.4 Interpretation.

The fact that Goemans-Williamson’s algorithm got full accuracy for all of the test cases is alarming, and an indication that our benchmarks were not the best at comparing the two (like we have hinted at before). The graphs we created are un-weighted and almost perfectly bipartite because of the planted balanced partition, and for such graphs the semidefinite relaxation is effective. The random hyperplane finds the optimal cut with probability 1.

6 Conclusion

The Goemans-Williamson is the best method for solving the maximum cut of graphs of sizes 5, 10, 20, 30, 50 where nodes are connected with a probability of 25%. This said it is not impossible that the deep learning approaches can bypass Goemans-Williamson in the future. The deep learning approaches do show some potential. Especially the execution speed of the transformer.

Areas where Deep Learning has potential: We can see that the inference speed in the transformer is always the fastest, and gets faster compared to the other two as the graph size grows. If we can successfully scale and improve the transformers decent results on the small graphs we could get a good alternative to Goemans-Williamson if we need a fast estimation. The fact that the cross-entropy is relatively high for the graphs of size 20,30,50 hints that we might be able to further train our models to increase their performance.

Improvements — Neural Networks: Several improvements could enhance the performance of the neural network models:

1. **More sophisticated row encoding** - performance
2. **Longer training duration** — especially for the Transformer, training for more epochs would likely benefit performance on larger graphs ($n = 20, 30, 50$).
3. **Larger training dataset** — generating more training would allow us to longer without overfitting.
4. **Better compilation and optimization**

- 5. **Weight pruning or compression** - inference speed
- 6. **Hyperparameter tuning** - performance

References

- [1] Ebubekir Buber and Banu Diri. Performance analysis and cpu vs gpu comparison for deep learning. *International Journal of Advances in Engineering and Technology*, 2020. Available at: https://www.researchgate.net/publication/334168063_Performance_Analysis_and_CPU_vs_GPU_Comparison_for_Deep_Learning.
- [2] Shenshen Gu and Yue Yang. A deep learning algorithm for the max-cut problem based on pointer network structure with supervised learning and reinforcement learning strategies. *Mathematics*, 2020.
- [3] Jean B. Lasserre. A MAX-CUT formulation of 0/1 programs.